

REGISTRO DE MOBILE - SEMANA 18

Resolução da Atividade: Explorando integrações – dominando API externas

1. Importância da Autenticação Correta

A autenticação garante a **segurança** e o **controle de acesso** entre o aplicativo da TravelWeather e a API WeatherPlus. Ela impede que pessoas mal-intencionadas façam requisições em nome do seu aplicativo, evita roubo de dados, permite que o provedor da API controle o limite de requisições por segundo (rate limiting) e ajuda a monitorar os custos de uso do serviço.

2. Importância de Compreender o Formato de Resposta (JSON/XML)

Cada API organiza as informações de um jeito. Compreender se o formato é **JSON** ou **XML** é vital para estruturar os modelos de dados (classes/objetos) no código do aplicativo mobile. Se o app tentar ler um campo que mudou de nome ou de formato, ocorrerá um erro de conversão (*parse error*), fazendo com que o aplicativo feche sozinho (*crash*).

3. Estratégias para Tratar Erros em APIs

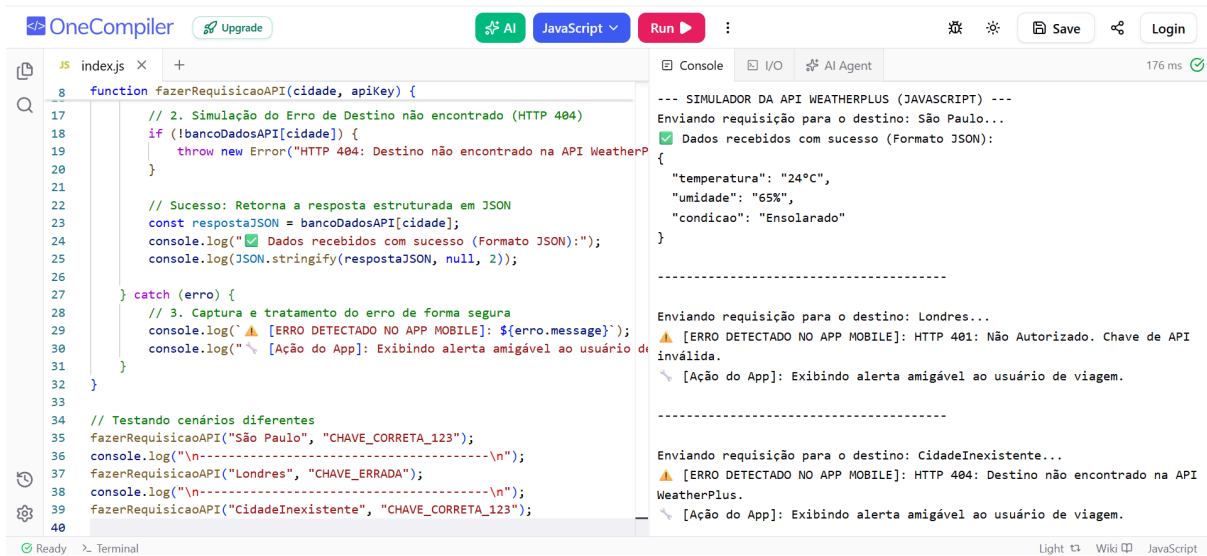
- **Blocos Try-Catch:** Capturar falhas de conexão antes que elas derrubem o aplicativo.
- **Verificação de Status HTTP:** Analisar os códigos de resposta do servidor (ex: **401** para erro de chave/autenticação, **404** para cidade não encontrada, **500** para erro interno do servidor).
- **Timeouts:** Definir um tempo máximo de espera para a resposta. Se o servidor demorar demais, a requisição é cancelada para não travar a tela do usuário.
- **Cache Local:** Salvar a última previsão climática de sucesso no dispositivo. Se a internet cair, o usuário ainda vê uma informação recente em vez de uma tela em branco.

4. Aplicação do Tratamento de Erros no Cenário TravelWeather

No aplicativo de viagens, o tratamento seria aplicado da seguinte forma:

- Se o usuário pesquisar uma cidade inexistente ou com erro de digitação (**HTTP 404**), o app exibe um aviso: *"Destino não encontrado. Verifique a grafia."*
- Se a chave da API falhar (**HTTP 401**), o sistema registra um log de erro interno para os desenvolvedores corrigirem a renovação do token.
- Se o usuário estiver sem internet, o aplicativo mostra uma mensagem amigável: *"Você está offline. Mostrando última previsão carregada."* junto com um botão de **"Tentar novamente"**.

Código teste JavaScript (OneCompiler):



```
function fazerRequisicaoAPI(cidade, apiKey) {
  // 2. Simulação do Erro de Destino não encontrado (HTTP 404)
  if (!bancoDadosAPI[cidade]) {
    throw new Error("HTTP 404: Destino não encontrado na API WeatherPlus");
  }

  // Sucesso: Retorna a resposta estruturada em JSON
  const respostaJSON = bancoDadosAPI[cidade];
  console.log("✅ Dados recebidos com sucesso (Formato JSON):");
  console.log(JSON.stringify(respostaJSON, null, 2));
} catch (erro) {
  // 3. Captura e tratamento do erro de forma segura
  console.log(`⚠️ [ERRO DETECTADO NO APP MOBILE]: ${erro.message}`);
  console.log(`🔗 [Ação do App]: Exibindo alerta amigável ao usuário de viagem.`);
}

// Testando cenários diferentes
fazerRequisicaoAPI("São Paulo", "CHAVE_CORRETA_123");
fazerRequisicaoAPI("Londres", "CHAVE_ERRADA");
fazerRequisicaoAPI("CidadeInexistente", "CHAVE_CORRETA_123");
```

Console Output:

```
--- SIMULADOR DA API WEATHERPLUS (JAVASCRIPT) ---
Enviando requisição para o destino: São Paulo...
✅ Dados recebidos com sucesso (Formato JSON):
{
  "temperatura": "24°C",
  "umidade": "65%",
  "condicao": "Ensolarado"
}

Enviando requisição para o destino: Londres...
⚠️ [ERRO DETECTADO NO APP MOBILE]: HTTP 401: Não Autorizado. Chave de API inválida.
🔗 [Ação do App]: Exibindo alerta amigável ao usuário de viagem.

Enviando requisição para o destino: CidadeInexistente...
⚠️ [ERRO DETECTADO NO APP MOBILE]: HTTP 404: Destino não encontrado na API WeatherPlus.
🔗 [Ação do App]: Exibindo alerta amigável ao usuário de viagem.
```

Perguntas de Consolidação:

1. Qual a importância de autenticar uma API corretamente em uma integração com serviços externos?

A autenticação correta é a **barreira de segurança** que protege tanto o seu aplicativo quanto o servidor externo. Ela é fundamental por três motivos principais:

- **Segurança e Controle:** Garante que apenas aplicativos autorizados acessem os dados, impedindo que pessoas mal-intencionadas façam requisições falsas ou roubem informações.
- **Monitoramento e Cobrança:** Permite que a empresa dona da API saiba exatamente quem está usando o serviço. Isso serve para monitorar o volume de acessos e cobrar corretamente caso o plano contratado seja pago por número de requisições. [1]
- **Prevenção de Abusos (Rate Limiting):** Evita que um único usuário ou aplicativo faça requisições em excesso (intencionalmente ou por erro de código), o que poderia sobrecarregar e derrubar os servidores da API. [1, 2]

2. Por que é essencial compreender o formato de resposta de uma API antes de consumi-la?

Compreender o formato de resposta (como **JSON** ou **XML**) é essencial para que o aplicativo consiga **interpretar e extrair** as informações recebidas de maneira correta (processo chamado de *parsing*). [1, 2]

- Se você não souber a estrutura exata da resposta (quais são as chaves, os nomes dos campos e se os valores são textos, números ou listas), o código do seu aplicativo mobile tentará ler dados inexistentes.

- Isso resultará em **erros de conversão** em tempo de execução, fazendo com que as informações fiquem em branco na tela ou, no pior dos cenários, fazendo o aplicativo fechar sozinho (*crash*) no celular do usuário.

3. Quais estratégias podem ser usadas para tratar erros ao consumir dados de API?

Para garantir que o aplicativo continue funcionando mesmo quando algo dá errado na internet, usamos as seguintes estratégias:

- **Blocos Try-Catch:** Envolver a requisição em uma estrutura que "captura" falhas de conexão de rede ou erros inesperados sem interromper a execução do aplicativo.
- **Validação de Códigos de Status HTTP:** Verificar o número retornado pelo servidor para tomar decisões automáticas (ex: **401** significa refazer o login/atualizar a chave; **404** significa avisar que o termo buscado não existe; **5xx** significa pedir para tentar mais tarde porque o servidor deles caiu). [1]
- **Definição de Timeouts:** Configurar um tempo limite de espera (ex: 10 segundos). Se a resposta demorar mais que isso devido a uma internet muito lenta, o app cancela a tentativa e avisa o usuário, evitando que a tela fique travada infinitamente.
- **Mecanismos de Cache Local:** Guardar as últimas informações baixadas com sucesso na memória do celular. Se uma nova requisição falhar por falta de sinal, o aplicativo exibe os dados salvos anteriormente em vez de uma tela vazia.

4. Como vocês aplicariam o conceito de “tratamento de erros” no cenário apresentado?

No aplicativo da **TravelWeather**, o tratamento de erros seria aplicado de forma prática para proteger a experiência do viajante:

- **Cenário de Destino Incorreto (Erro 404):** Se o usuário digitar o nome de uma cidade que a API WeatherPlus não reconhece, o aplicativo interceptará o erro e exibirá uma mensagem amigável: *"Não encontramos previsões para este destino. Por favor, verifique se o nome está correto."*
- **Cenário de Falha de Conexão (Falta de Internet):** Caso o viajante tente olhar a previsão do tempo dentro de um avião ou em um lugar sem sinal, o aplicativo capturará a falha de rede, mostrará um aviso informando que ele está offline e exibirá a última previsão climática que ficou salva no cache do celular.
- **Cenário de Chave Expirada (Erro 401):** Se a chave de API do aplicativo falhar por problemas de renovação de contrato, o app não mostrará códigos técnicos assustadores para o usuário; em vez disso, enviará um alerta interno para a equipe de desenvolvimento da TravelWeather e exibirá uma mensagem temporária na tela: *"Estamos atualizando nossos sistemas de clima. Tente novamente em instantes."*